Disjoint Set Union & Minimum Spanning Trees

Near-constant-time connectivity, Kruskal and Prim

Learning goals

- Implement **DSU** (union-find) with path compression and union by size
- Explain why it runs in O(α(n)) amortised per operation
- Use DSU for **online connectivity** and component statistics
- Find a **minimum spanning tree** with **Kruskal** and with **Prim**
- Prove MST correctness with the **cut property**
- Recognise "offline + sort + DSU" problems

Motivation: Road Construction (CSES 1676)

- n cities, no roads. m roads are built one by one
- After **each** road: number of components, size of the largest one
- $n, m \leq 10^5$

Re-running BFS after each road: $O(m \cdot (n + m)) \approx 10^{10}$ ×

We need a structure supporting:

- `find(x)` : which group is x in?
- `unite(a, b)` : merge the groups of a and b

DSU: forest of parent pointers

Each group is a rooted tree; the **root** is the group's representative.

```
p: [1]=1 [2]=1 [3]=1 [4]=4 [5]=4          1        4
                                         / \       |
                                        2   3      5
```

- `find(x)`: follow `p[]` until `p[r] == r`
- `unite(a, b)`: `p[find(b)] = find(a)`
- Same group ⇔ `find(a) == find(b)`

Naive version: trees can become paths → O(n) per find ×

Optimisation 1: union by size

Attach the **smaller** tree under the **larger** root.

```
if (sz[a] < sz[b]) swap(a, b);
p[b] = a;
sz[a] += sz[b];
```

A vertex's depth grows only when its tree at least **doubles** in size → depth ≤ $\log_2 n$.

Bonus: `sz[root]` = component size, for free.

Optimisation 2: path compression

During `find`, point visited nodes closer to the root.

```
before find(5):        after (full compression):
1 ← 2 ← 3 ← 4 ← 5       1 ← 2, 1 ← 3, 1 ← 4, 1 ← 5
```

Path halving (iterative, no recursion):

```
int find(int x) {
    while (p[x] != x) { p[x] = p[p[x]]; x = p[x]; }
    return x;
}
```

Both together: **O(α(n))** amortised; α(n) ≤ 4 for any n you'll ever see.

DSU code

```
struct DSU {
    vector<int> p, sz;
    DSU(int n) : p(n), sz(n, 1) { iota(p.begin(), p.end(), 0); }
    int find(int x) {
        while (p[x] != x) { p[x] = p[p[x]]; x = p[x]; }
        return x;
    }
    bool unite(int a, int b) {
        a = find(a); b = find(b);
        if (a == b) return false;
        if (sz[a] < sz[b]) swap(a, b);
        p[b] = a; sz[a] += sz[b];
        return true;
    }
};
```

`unite` returns whether a merge happened — very handy.

Road Construction solution

```
DSU d(n + 1);
int comps = n, best = 1;
while (m--) {
    int a, b; cin >> a >> b;
    if (d.unite(a, b)) {
        comps--;
        best = max(best, d.sz[d.find(a)]);
    }
    cout << comps << " " << best << "\n";
}
```

O((n + m) α(n)). The largest component only ever grows → a running max is enough.

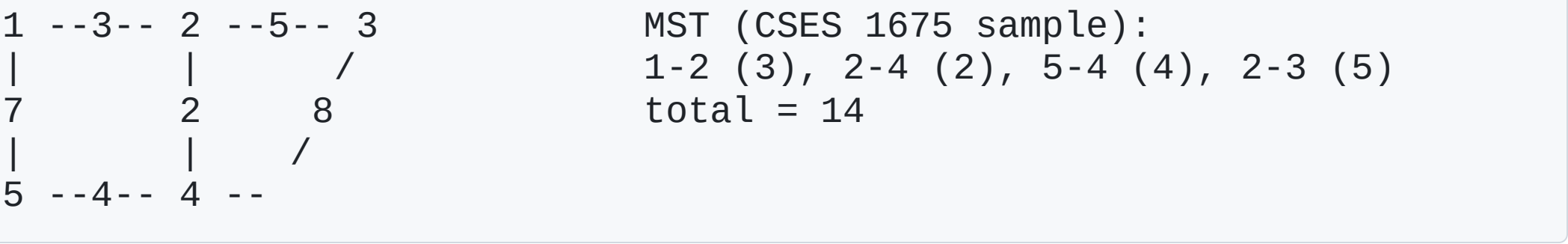
What DSU can and can't do

✓ merge groups, ask same-group, size, any **mergeable** aggregate (sum, min, count of edges...)
✓ detect a cycle when adding an undirected edge (`unite` returns false)
× **split** groups / delete edges (go offline and reverse time!)
× give the actual path between two vertices

Trick: deletions only? **Process in reverse** → they become additions.

Minimum spanning tree

Connected, undirected, weighted graph. A **spanning tree** uses n − 1 edges and connects all vertices. **MST** = spanning tree with minimum total weight.



Uses: cheapest network, clustering, bottleneck paths, approximations.

The cut property

Cut = split of V into S and V \S.

> For any cut, a minimum-weight edge crossing it belongs to **some** MST.

Proof (exchange): take an MST T without that edge e = (u, v). T has a path u → v that crosses the cut via some edge f. w(e) ≤ w(f), so T − f + e is a spanning tree no heavier than T. ∎

Both Kruskal and Prim are "repeatedly add a min edge across some cut".

Kruskal's algorithm

1. Sort edges by weight
2. For each edge (a, b): if a and b are in different components, take it

```
sort(edges.begin(), edges.end());          // (w, a, b)
DSU d(n + 1);
long long total = 0; int used = 0;
for (auto [w, a, b] : edges)
    if (d.unite(a, b)) { total += w; used++; }
if (used != n - 1) → graph is disconnected
```

Correct by the cut property (cut = component of a vs the rest).

O(m log m) for the sort; DSU part ~ O(m).

Kruskal trace (CSES 1675 sample)

edge (w)	components before	take?
2–4 (2)	{1}{2}{3}{4}{5}	✓
1–2 (3)	{1}{2,4}{3}{5}	✓
5–4 (4)	{1,2,4}{3}{5}	✓
2–3 (5)	{1,2,4,5}{3}	✓
1–5 (7)	{1,2,3,4,5}	× cycle
3–4 (8)		× cycle

Total = 2 + 3 + 4 + 5 = **14**

Prim's algorithm

Grow **one** tree from vertex 1; repeatedly add the cheapest edge leaving it.

```
pq.push({0, 1});
while (!pq.empty()) {
    auto [w, u] = pq.top(); pq.pop();
    if (inTree[u]) continue;              // stale
    inTree[u] = true; total += w; added++;
    for (auto [v, c] : adj[u])
        if (!inTree[v]) pq.push({c, v});
}
```

Looks like Dijkstra — but the key is the **edge weight**, not distance.

O(m log m). Disconnected ⇔ `added < n`.

Kruskal vs Prim

	Kruskal	Prim (heap)
Input form	edge list	adjacency list
Time	$O(m \log m)$	$O(m \log m)$
Extra structure	DSU	priority queue
Dense graph ($m \approx n^2$)	sort n^2 edges	$O(n^2)$ array version is best
Partial forest / offline queries	natural	awkward

Contest default: **Kruskal** — shortest code, and the DSU is reusable.

MST facts worth knowing

- Distinct weights ⇒ MST is **unique**
- Max spanning tree: sort descending (or negate)
- MST minimises the **maximum** edge on paths too (minimax / bottleneck path)
- Forced edges: give them weight $-\infty$ (unite them first)
- Minimum spanning **forest** on disconnected graphs: Kruskal just works
- Negative weights are fine (unlike Dijkstra)

Offline DSU: Path Queries (CF 1213G)

Tree with weighted edges; query q: how many pairs (u, v) have **max edge on path ≤ q**?

- Sort edges by weight, sort queries by q
- Add edges with w ≤ q; merging sizes s_1 and s_2 creates $\mathbf{s_1 \cdot s_2}$ new pairs
- Answer = running total

O((n + m) log) — the Kruskal sweep as a counting tool.

Offline DSU code (core)

```
sort(edges.begin(), edges.end());              // (w, u, v)
sort(qs.begin(), qs.end());                    // (q, original index)
long long pairs = 0; size_t e = 0;
for (auto [q, idx] : qs) {
    while (e < edges.size() && edges[e][0] <= q) {
        int a = d.find(edges[e][1]), b = d.find(edges[e][2]);
        pairs += 1LL * d.sz[a] * d.sz[b];
        d.unite(a, b); e++;
    }
    ans[idx] = pairs;
}
```

Complexity summary

Operation	Time
DSU find / unite (both optimisations)	$O(\alpha(n))$ amortised
DSU with only one optimisation	$O(\log n)$
Kruskal	$O(m \log m)$
Prim with heap	$O(m \log m)$
Prim with array (dense)	$O(n^2)$

Common pitfalls

- Comparing `p[a] == p[b]` instead of `find(a) == find(b)`
- Reading `sz[x]` for a non-root x
- `unite` without calling `find` first → corrupts trees
- Forgetting to check that the MST has n − 1 edges (disconnected)
- MST total in `int` → overflow; use `long long`
- Recursive `find` without union by size → deep recursion on paths
- Using Prim's Dijkstra-style key `dist[u] + w` — that's **wrong** for MST

Practice problems

- CSES 1676 Road Construction · CSES 1675 Road Reparation
- CF 1213G Path Queries · CF 25D Roads not only in Berland
- AtCoder Library Practice Contest A (Disjoint Set Union)
- Kattis `minspantree` · Kattis `unionfind`
- CF 1513D GCD and MST · USACO 2016 Open Gold "Closing the Farm"
- CSES 1677 Network Breakdown (reverse time!)

Summary

- DSU = forest of parent pointers + union by size + path compression → ~O(1)
- Use it for incremental connectivity, cycle detection, component sizes
- MST via cut property; **Kruskal** = sort + DSU, **Prim** = grow with a heap
- Offline: sort events, sweep, merge — and reverse time to turn deletions into unions
- Next week: **Trees** — traversals, diameter, subtree sizes and tree DP